Bound Methods in ISO C via Runtime-Generated Trampolines

A technical account of the CMT library's object/inject mechanism

Teoman Deniz - 2026/08/15

Abstract

Standard C has no notion of a bound method. A function pointer stored in a `struct` carries no memory of which instance it belongs to, so every C "object system" is forced to pass the instance explicitly - `obj->method(obj, args)` - or to fake binding with closures that most C compilers don't actually support portably.

This document describes an alternative: generating a small, per-instance stub of executable machine code at construction time. The stub's only job is to load the instance pointer into a fixed CPU register and jump to the real implementation. Because the callee recovers `self` from that register instead of from its argument list, call sites can look like genuine bound method calls - `obj->method(args)` - while the compiler is never told anything unusual is happening. This is the mechanism behind the CMT library's `object`/`new`/`object__inject` family of macros.

The technique is a **trampoline**, in the same sense the term is used for libffi closures, GCC nested functions, and Objective-C's IMP dispatch: a tiny piece of glue code, generated on the fly, whose only purpose is to bind context before handing off control.

Contents

1. The Problem: C Has No Bound Methods

In C, a member function pointer is just a function pointer. Nothing connects it to the struct it was assigned from:

```
struct object {
    void (*add)(int);
};
```

If `add` needs to touch the fields of `struct object`, it needs a pointer to that specific instance. Since C function pointers carry no hidden state, the instance has to travel through an explicit parameter:

```
struct object {
    void (*add)(struct object *self, int value);
};

obj->add(obj, 42);   /* the caller must repeat the instance by hand */
```

This is exactly how C++ compiles member functions under the hood - every non-static method silently gains a leading `this` parameter. C++ hides that parameter from the caller because the compiler inserts it automatically. C has no such privilege: the call site is what it is, and the instance has to be passed by hand, every time, at every call.

Two consequences follow:

Redundancy. The caller already "has" `obj` (that's how it got the function pointer); it has to say so again.

Error surface. Passing the wrong instance pointer to the wrong stored function pointer is a silent bug - the types usually still match.

A true bound method would let the pointer *already know* which instance it belongs to, the way a closure captures its environment. C has no closures. But the CPU underneath doesn't care about that limitation - a bound method is just a piece of code that happens to know an address before it starts running, and code that knows an address before it starts running is exactly what a compiler generates from a constant.

2. The Core Idea: Compile the Binding, Not the Behavior

Instead of storing a pointer to a fixed function and passing the instance as data, the library generates a distinct, tiny function *per instance* whose body has the instance address baked in as an immediate value. This generated function has one job: place that hardcoded address somewhere the real implementation can find it, then jump - not call, jump - into that implementation.

Conceptually, for one `test` instance, the library synthesizes machine code equivalent to:

```
mov  rax, <address of test>   ; hardcode which instance this is
mov  r11, <address of add>    ; hardcode which function to run
jmp  r11                      ; hand off control, no return address pushed
```

This snippet is the trampoline. It is written into freshly allocated executable memory, and its address - not `add`'s address - is what gets stored in `test.add`. When the caller writes `test.add(42)`, it is unknowingly calling the trampoline, which vanishes into `add` a couple of instructions later. `add` never sees the trampoline; it only sees `RAX` holding the instance pointer, as if that pointer had always been there.

This is the same family of technique used by:

libffi closures, which generate executable stubs so a C function pointer can carry bound context across an API boundary.

GCC/Clang nested function trampolines (`-fnested-functions` / `__builtin_trampoline_iy`), which do exactly this to let a nested function capture its enclosing frame.

Objective-C's IMP, where method dispatch resolves to a code pointer that expects `self` in a known location before it runs.

The library's contribution is applying this pattern directly, in portable, readable C macros, layered over a small amount of unavoidable architecture-specific assembly.

3. Why a Register, and Why RAX

The generated stub needs to communicate the instance pointer to the target function without changing the target function's calling convention or its parameter list - `add(int value)` must keep looking exactly like a plain function that takes one `int`. Registers are the only channel that satisfies this: they aren't part of the C parameter list, so slipping a value into one before the jump doesn't perturb the signature the compiler expects.

RAX specifically is used because, per the System V AMD64 ABI, it is a caller-saved scratch register with no fixed argument role - nothing on the call path expects it to hold anything in particular, so the library is free to claim it as an out-of-band channel. On entry to `add`, before the compiler-generated prologue touches anything, `object__connect` reads `RAX` straight out and treats it as `THIS`:

```
#define OBJECT__CONNECT(OBJECT_TYPE_NAME) \
    register PTR                LOCALMACRO__SELF__; \
    GET_RAX(LOCALMACRO__SELF__); \
    OBJECT OBJECT_TYPE_NAME    *const THIS = \
        (OBJECT OBJECT_TYPE_NAME *)LOCALMACRO__SELF__
```

The trampoline places the pointer in `RAX`; `object__connect` takes it back out. Everything in between - the `jmp`, not `call` - is what keeps the handoff transparent: because it's a jump rather than a call, no new stack frame or return address is introduced, and register state (including `RAX`) survives into the callee exactly as the trampoline left it.

32-bit and 16-bit variants follow the same idea with architecture-appropriate registers and instructions (`EAX`/`EDX` with `mov`/`jmp`, or a `push`+`ret` sequence on 16-bit, where jumping through a register is less convenient).

4. Anatomy of the Macro Chain

4.1 Declaring the shape: object / OBJECT

```
#define OBJECT struct
#define object struct
```

Purely cosmetic - `object` reads better than `struct` when the whole point is to imitate an object system, but it compiles to an ordinary struct declaration. No hidden vtable, no hidden fields are injected here.

4.2 Recovering self inside a method: object_connect

Shown above. Every function meant to act as a "member" starts by calling `object__connect(type_name)`, which declares a local `this` (or `THIS`) typed as a pointer to that struct, sourced from `RAX`. This macro is what makes `add`'s body able to write `this->value += value` despite `add`'s own signature never mentioning a struct at all.

4.3 Constructing an instance: NEW / new / OBJ / obj

```
#define NEW(OBJECT_NAME, VARIABLE_NAME) \
    OBJECT OBJECT_NAME    VARIABLE_NAME; \
    extern void            OBJECT_NAME(); \
    \
    SET_RAX(&VARIABLE_NAME); \
    OBJECT_NAME
```

`new(test_object_type, test)(42)` expands to: declare the struct storage, forward-declare the constructor function, load `RAX` with the new object's address, then call the constructor - which is really just another function that begins with `object__connect` to recover that same address. The constructor body (`test_object_type` in the example) is written like a regular function taking `(int start_var)`; it never sees an explicit struct pointer parameter either. It uses the same `RAX` relay that member calls use, just triggered manually by the macro instead of by a generated trampoline.

4.4 Binding a function to an instance: object_inject

This is where the trampoline is actually built:

```
#define OBJECT__INJECT(TARGET, SOURCE) \
    do \
    { \
        register LET    INDEX; \
        BYTE            __OP__[LOCALMACRO__FUNCTION_SIZE]; \
        \
        INDEX = 0; \
        LOCALMACRO__INJECTOR(OBJECT_PTR, SOURCE) \
        (TARGET) = ALLOC_EXE(__OP__, LOCALMACRO__FUNCTION_SIZE); \
    } \
    while (0)
```

Step by step:

1. `__OP__` is a small stack buffer, exactly `LOCALMACRO__FUNCTION_SIZE` bytes - the precomputed size of the target architecture's trampoline instructions (e.g. `movabs` + `movabs` + `jmp` on x86-64).

2. `LOCALMACRO__INJECTOR(OBJECT_PTR, SOURCE)` expands to the architecture-specific assembly macros (`_MOVABS_RAX_`, `_MOVABS_R11_`, `_JMP_R11`, ...) that encode `OBJECT_PTR` and `SOURCE` as immediates directly into `__OP__`, byte by byte, advancing `INDEX` as they go.

3. `ALLOC_EXE(__OP__, size)` copies those bytes into freshly allocated **executable** memory and returns its address.

4. That address - the trampoline's entry point - is assigned to the target member (`(OBJECT_PTR)->TARGET`).

`object__inject(this->add, add)` - bind the field named `add` to the function named `add`. The two-argument and three-argument forms exist so a struct field can be bound to a differently-named function, as in the example:

```
object__inject(this->free, free_self);   /* field "this->free" -> function free_self
*/
```

This is why calling convention mismatches - like *`test_object_type`*'s own struct field also being named free, a name that collides with `<stdlib.h>`'s `free` - can be routed around simply by naming the underlying function something else.

4.5 Releasing a trampoline: object_eject

```
#define OBJECT__EJECT(MEMBER_NAME) FREE_EXE(THIS->MEMBER_NAME)
```

Because each bound member is backed by real allocated executable memory (not just a plain function pointer into static `.text`), it must be freed like any other heap resource. Forgetting to call `object__eject` on every injected member leaks executable pages, one per instance, per method - worth flagging prominently in end-user documentation, since it is the one part of this pattern that behaves unlike ordinary C function pointers.

5. Full Path of One Call

Tracing `test.add(42)` from the example program:

1. At injection time (inside the constructor, via `object__inject(add)`): a trampoline is generated that hardcodes `&test` and the address of the real `add` function, and its address is stored in `test.add`.

2. At call time: `test.add(42)` is, to the compiler, an ordinary indirect call through a function pointer of type `void(*)(int)`. It pushes `42` per the normal calling convention and jumps to whatever address is in `test.add` - which is now the trampoline, not `add` itself.

3. Inside the trampoline: `RAX` is loaded with `&test`, `R11` is loaded with `&add`, and control jumps to `R11`. Because this is a `jmp` and not a `call`, no stack frame is added - the trampoline is invisible to unwinding and to `add`'s own prologue.

4. Inside add: the first thing the generated code does (via `object__connect(test_object_type)`) is read `RAX` and reinterpret it as `THIS`, a `struct test_object_type *`. From here, `this->value += value` behaves exactly as if `add` had been declared `void add(struct test_object_type *this, int value)` and called as `add(&test, value)` - except the call site never had to say so.

The `42` argument itself never touches the trampoline at all; it travels through the normal ABI argument-passing path (a register or the stack, depending on architecture) exactly as it would for any ordinary function call. Only the *instance* pointer is smuggled through the side channel.

6. Per-Instance Cost and Isolation

Each call to `object__inject` allocates one small block of executable memory and copies a handful of bytes into it. This has a direct consequence worth stating plainly: **every instance carries its own private trampoline per bound method**, not a shared one. `test` and `test2` in the example each get their own `add` trampoline, hardcoding `&test` and `&test2` respectively, even though both ultimately jump into the same underlying `add` function.

This is what makes `test.add(42)` and `test2.add(0)` independently correct without either call needing to say which instance it's operating on - the disambiguation was already baked into the trampoline at construction time, rather than resolved at the call site. It is also the direct cost of the technique: N instances with M bound methods each means N x M allocated executable stubs, each needing an explicit `object__eject`.

7. Safety Notes

W^X / DEP compliance. The `ALLOC_EXE` step must write into a page and later execute from it. On modern systems, writable and executable permissions on the same page are not simultaneously granted by default (W^X); `ALLOC_EXE` is expected to handle allocate -> write -> mark executable as separate steps (e.g. via `mmap` + `mprotect`, or platform-equivalent), rather than requesting a page that is simultaneously writable and executable. This detail lives entirely inside `LIB/MEMORY.H` and is invisible to code using the macros - but it is the load-bearing assumption that keeps the whole scheme legal on hardened operating systems.

Architecture coverage. The trampoline bodies are written directly in assembly per architecture (`__SYSTEM_64_BIT__`, `__SYSTEM_32_BIT__`, `__SYSTEM_16_BIT__` under `__CPU_INTEL__`), selected at preprocessing time. Ports to non-Intel architectures (ARM, PowerPC) require a new `LOCALMACRO__INJECTOR` definition and new `SIZEOF_*` constants, but do not require any change to the macros described in Section 4 - the architecture boundary is fully isolated to the assembly layer.

Lifetime discipline. Because each bound method is a heap-allocated executable stub rather than a static function address, its lifetime must be managed explicitly with `object__eject`. This is the one place the abstraction is not "free" - it trades the ergonomics of `obj->method()` for an extra teardown obligation that a plain function-pointer struct would never have.

Strict aliasing / calling convention. The generated stub must exactly reproduce the calling convention the target function expects (register usage, stack alignment) or the jump will corrupt arguments. This is why the trampoline size and instruction sequence (`LOCALMACRO__FUNCTION_SIZE`, `LOCALMACRO__INJECTOR`) are defined per-architecture rather than assumed portable.

8. Summary

Concern	Conventional C	This technique
Call site	obj->method(obj, args)	obj->method(args)
Instance binding	Explicit parameter	Baked into generated code
Storage per method	One function pointer (shared)	One executable stub (per instance)
Teardown	None needed	object__eject required
Portability	Fully portable ISO C	Requires per-architecture assembly for the trampoline body

The technique does not extend the C type system, and it does not require a non-standard compiler extension - the "magic" is confined to a handful of architecture-specific instruction sequences and a documented, deliberate use of one otherwise-unspecified scratch register as a communication channel between generated code and hand-written code. Everything else - the macros, the struct declarations, the call sites - is expressible, and readable, in plain ISO C.